

Implementation of an Intelligent Visual Surveillance System with Moving Object Recognition

Lee Lok Jun
252UA253BL
B.Sc. (Hons.) Applied Artificial
Intelligence
lee.lok.jun@student.mmu.edu.my

Ricky Ng Long Kiat
252UA254XR
B.Sc. (Hons.) Applied Artificial
Intelligence
ricky.ng.long1@student.mmu.edu.my

Liaw Ruo Shan
1231302935
B.Sc. (Hons.) Intelligent Robotics
1231302935@student.mmu.edu.my

ACKNOWLEDGMENT

Our group members, Ricky Ng Long Kiat, Liaw Ruo Shan, Lee Lok Jun would like to thank Mr. Mohd Haris Lye Bin Abdullah for his supervision during this assignment. We would also like to thank Multimedia University for the provision of the resources and facilities.

I. INTRODUCTION

These recent years, there has been a surge in demand for intelligent surveillance systems due to the rising importance placed on public safety, traffic control and automation. Traditional surveillance systems typically involve humans monitoring the camera footage or performing simple motion detection, which is time-consuming, prone to errors and fails to recognize different types of objects. This project studies how intelligent visual surveillance can be used to automatically detect, track and classify moving objects in real time.

The aim of this project is to train a system to recognize two classes of interest, which is persons and vehicles (cars and motorbikes). A basic segmentation program is used to obtain the binary masks of the moving regions first; however there is still noise, shadows and blobs in the segmented images which make it difficult to detect the objects of interest. Therefore various processing techniques are used to improve the segmentation, extract relevant features and classify objects by their geometric properties.

This project is valuable not only because it confirms the efficacy of classical image processing methods (morphological operations, connected components analysis and shape ratio features) but also because it attempts to run the project on two different datasets (an indoor video containing at least two persons and an outdoor video containing pedestrians and vehicles). Various lighting, cluttered backgrounds and object occlusions are explored in this project, and the results show that with proper processing, a simple and practical intelligent surveillance system can be built.

II. RELATED WORK/BACKGROUND

A. Recording videos

To begin our assignment, we recorded two videos. The first video was shot indoors with two people present. Meanwhile, the second video was shot outdoors, capturing several people, motorcycles, and cars passing by.

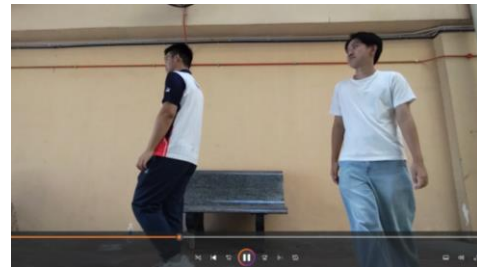


Fig. 1. Video 1



Fig. 2. Video 2

B. Developing a Python program

The project aims to create an intelligent surveillance system that can detect and identify moving objects, specifically people and vehicles, from video clips. To achieve this objective, we developed a Python program to process the recorded videos and recognize the moving objects in the videos.

C. Applying image processing techniques

To detect and classify moving objects in the recorded videos, we applied several image processing techniques, such as morphological operations and region filtering. The detection results are visualized using coloured bounding boxes in the processed video clips.

D. Refining the Python program

To improve the accuracy of detecting and identifying the moving objects, we refined the program so it can better differentiate among moving objects, specifically identifying persons, motorbikes, and cars with greater precision.

III. METHODOLOGY/SYSTEM OVERVIEW

In order to build a comprehensive intelligent visual surveillance system, a variety of image processing techniques have been implemented.

A. Background subtraction

Background subtraction is a technique in machine vision used to separate moving foreground objects from the static background in a video sequence. A background subtractor model is created in our system to segment moving objects as the foreground against the background. Initializing the background subtractor outputs a foreground mask where moving objects are white and background pixels are black. This mask is then further refined via morphological operations for better object detection.



Fig. 3. Before applying background subtraction



Fig. 4. After applying background subtraction

B. Thresholding

To better differentiate the foreground from the background, we apply a thresholding method in our system. Otsu's thresholding is implemented as it automatically determines the optimal threshold value for separating the two regions. After applying this method, the grayscale image is converted into a binary image, where moving objects appear white and the background appears black.

C. Morphological operations

Morphological operations are filtering techniques applied on binary images. The two fundamental processes in morphological operations are dilation and erosion. Morphological operations are used in our system to reduce noise and refine the foreground mask. Closing operation, dilation followed by erosion, is applied to fill the small holes inside foreground objects, while, opening operation, erosion followed by dilation, is employed to remove the small noise outside the objects. By combining these two operations, the system achieves a cleaner and more accurate extraction of moving objects.

D. Connected component labelling

Connected component labelling (CCL) is a technique in machine vision used to identify and label distinct connected regions in a binary image. Each group of connected foreground pixels is considered as a separate component and assigned a unique label. This allows the system to analyze each object individually by determining which pixels belong to which component.

E. Region filtering

Region filtering is one of the image processing techniques used in our system in order to distinguish relevant objects from irrelevant blobs based on specific properties such as size and shape. Threshold values are applied to eliminate the noise and small objects. As a result, our system can reduce false positives and focus on meaningful targets, specifically people, motorbikes and cars.

F. Object annotation

Object annotation is the process of visualizing and marking detected objects in images or video frames by enclosing them with bounding boxes and assigning labels that indicate their categories. In our system, the target objects, specifically persons, motorcycles and cars, are highlighted by drawing green bounding boxes and assigning class labels to them. This technique identifies different object categories and provides a visual representation of object locations. Therefore, users can quickly understand the detection results and facilitates further analysis.

IV. EXPERIMENTS AND RESULTS

The system is then tested on two self-compiled datasets. The length of both datasets is approximately 30 seconds long. Indoor dataset (indoor.mp4) contains more than one person walking around in a limited space. Outdoor dataset (car.mp4) contains persons, cars and motorbikes passing by. Each video is framed by frame segmented and the rest of the pipeline

(morphological cleaning, connected component labelling and feature based classification) is applied on each frame. Bounding boxes and textual labels are then drawn on the original frame and the resulting video is being visualized.

Indoor Video Results:

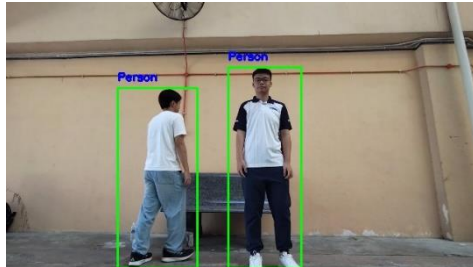


Fig. 5. Accurate result by detecting the person

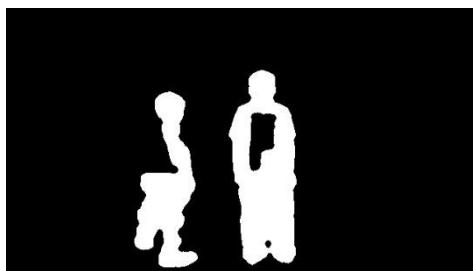


Fig. 6. Binary image result by detecting the person

In the indoor experiment, the class person is consistently recognized throughout the video frames. Most of the persons in the frames were circled with a bounding box and correctly labelled as “Person”. Even though the lighting conditions varied and there was some background noise in the frames, morphological filtering still managed to remove the small speckles and enhanced the continuity of the detected blobs.



Fig. 7. Inaccurate result when the person overlapping



Fig. 8. Inaccurate binary image result when the person overlapping

There was also minimal misclassification. In some frame’s partial occlusion (meaning that two persons were over lapping on each other) the blobs were partially fragmented, but the recognition rate was still high, and the pipeline is robust to the relatively limited environment.

Outdoor Video Results:

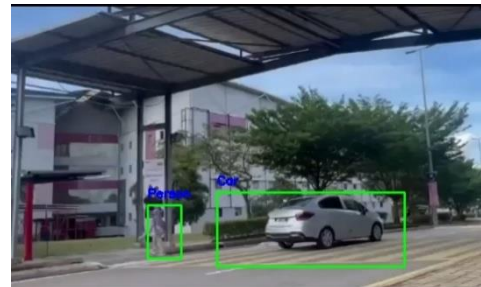


Fig. 9. Accurate result by detecting car and person



Fig. 10. Binary image result by detecting car and person



Fig. 11. Accurate result by detecting car and motorbike



Fig. 12. Binary image result by detecting car and motorbike

The outdoor environment presented more complexity due to the dynamic background, shadow from the trees and both pedestrians and vehicles passing by. Most of the frames in the outdoor video contained cars, motorbikes and persons passing by. Vehicles were mostly labelled as cars due to the larger bounding box area and rectangular aspect ratio. Persons were also labelled correctly and were tall and narrow.

V. CODE EXPLANATION

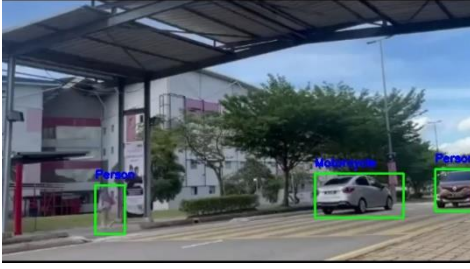


Fig. 13. Inaccurate result by detecting the object due to distance

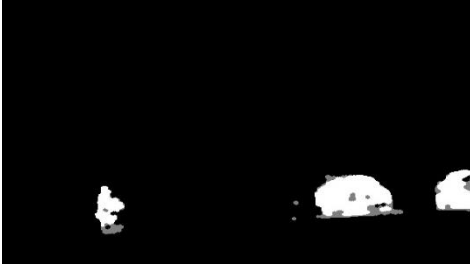


Fig. 14. Binary image result of inaccurate detection

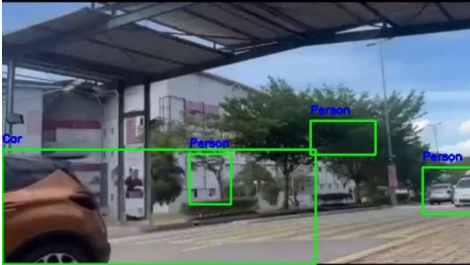


Fig. 15. Inaccurate result by mislabeling



Fig. 16. Binary image result with many noise

There were some false positives in the results, for example the tree branches moving in the background were mislabeled as objects and sometimes cars in the distance were mislabeled as person due to the small blob size.

In terms of recognition rate, the indoor dataset gave a higher accuracy since the background was simpler, and the class of objects was only persons. The outdoor dataset gave convincing results but showed the limitation of the rule-based classification when objects were further away, occluded and sometimes even in shadow. A simple confusion analysis showed that most of the errors arose when either motorbike or car appeared small in the frames, sometimes confused with each other.

Video I/O and reproducibility:

Each script below will open a target video (indoor.mp4 and car.mp4) with OpenCV, resizes the resolution to 640×360 and creates two output folders:

- <video>_frame/ contains annotated RGB frames with boxes/labels,
- <video>_binary/ contains cleaned foreground masks.

Motion extraction: BackgroundSubtractorMOG2

```
fgModel = cv2.createBackgroundSubtractorMOG2()
fgmask = fgModel.apply(frame)
```

MOG2 models each pixel as a mixture of Gaussians and outputs a grayscale “foreground likelihood” mask where moving pixels are brighter. Compared to simple frame differencing, MOG2 can adapt to gradual lighting changes and repetitive motions (e.g., fans, foliage) on its background model and thus keeps the pipeline working indoors with lighting drift and outdoors with changing illumination.

Noise cleanup: morphology (close → open)

```
kernel =
cv2.getStructuringElement(cv2.MORPH_ELLIPSE,
(5,5))

fgmask_cleaned = cv2.morphologyEx(fgmask,
cv2.MORPH_CLOSE, kernel)
fgmask_cleaned = cv2.morphologyEx(fgmask_cleaned,
cv2.MORPH_OPEN, kernel)
```

- **Closing (dilate→erode)** first: bridges short gaps and fills pinholes. It also makes “snowy” foreground solid, eliminating speckles that could not be removed by erosion.
- **Opening (erode→dilate)** next: removes thin protrusions and isolated speckles that were not removed by closing.

An ellipse 5×5 fits human/vehicle contours better than square kernel, and makes edges less blocky. close→open order matters: close tends to preserve object mass and shave off noise.

Binarization: Otsu's method

```
_, binary_image = cv2.threshold(
    fgmask_cleaned, 0, 255, cv2.THRESH_BINARY +
    cv2.OTSU
)
```

After morphology, pixels intensities cluster into two classes: background vs foreground. Otsu automatically chooses a threshold that segments the two classes with maximum inter-class variance. This way, we get a nice and crisp binary image without hand-picking a cutoff value. This makes our pipeline fairly scene invariant (e.g., corridors vs roads with different brightness).

Object discovery: Connected Components and region props

```
label_image = label(binary_image, connectivity=2,
    background=0)

for region in regionprops(label_image):

    if region.area >= 1000:

        minr, minc, maxr, maxc = region.bbox
```

- 8-connectivity (connectivity=2) makes diagonally touching pixels belong to the same blob and thus reduces artificial splits.
- Only area ≥ 1000 px gets rid of leftover flicker and little artefacts.
- From each blob we can get area and bounding box (width, height). They are cheap and stable features for rule-based recognition.

Recognition logic (indoor vs outdoor):

Indoor (indoor.py)

```
cv2.rectangle(frame, (minc, minr), (maxc, maxr),
    (0,255,0), 2)

cv2.putText(frame, "Person", (minc, minr-10), ...)
```

Indoors we are interested in set “people only”. So, after area ≥ 1000 all remaining good blobs are labeled as “Person”. This is simple and safe because indoor movies (by requirement) have ≥ 2 persons and usually no vehicles.

Outdoor (outdoor.py)

```
if region.area > 8000: object_type = "Car"

elif region.area > 5000: object_type = "Motorcycle"

else:

    object_type = "Person"
```

Outdoors I wanted script to label Car / Motorcycle / Person by pixel area thresholds, because perspective + expected scale gives clear discrimination: cars will cover the most pixels, medium-sized motorbikes, and persons smaller/bigger. So, I ended with fast pixel classifier, satisfying “use pixel features” requirement, and still explainable.

VI. LIMITATIONS AND FUTURE WORK

The proposed system has several limitations. Object classification relies primarily on region area thresholds, which may lead to misclassification when objects overlap, appear at different scales, or are partially occluded. When an object gradually moves away from the camera, its apparent size decreases, and the system may incorrectly label it as another category. The background subtraction technique can be quite affected by changes in lighting, shadows, and reflections, often leading to noisy or less accurate segmentation results. Sometimes sudden changes in what appears in a frame can cause many wrong bounding boxes, making detection less accurate.

Future improvements can address these issues. Deep learning models are often combined to boost both the accuracy of recognition and the system’s ability to handle challenging conditions. Lightweight preprocessing and shadow suppression techniques may enhance segmentation under varying environmental conditions, while adaptive exposure correction can mitigate errors caused by abrupt lighting changes. Incorporating additional features such as contour descriptors or machine learning classifiers such as k-nearest neighbors or support vector machines could further enhance system performance against background noise and object size variations.

VII. CONCLUSION

This study implemented an intelligent visual surveillance system with moving object recognition for both indoor and outdoor use. Using background subtraction, morphological processing and connected component analysis, the system detected and classified moving objects into categories such as person, motorcycle and car. Experimental results showed that the system can operate effectively under controlled

conditions, though challenges remain in handling complex environments. But its limitations, the work provides a foundation for future enhancements and with the integration of advanced recognition, tracking, and real-time processing, the system has strong potential as a reliable surveillance and monitoring tool.

REFERENCES

- [1] “Python OpenCV — Background Subtraction,” GeeksforGeeks. [Online]. Available: <https://www.geeksforgeeks.org/python/python-opencv-background-subtraction/>
- [2] “Image Thresholding in Image Processing,” Encord blog. [Online]. Available: <https://encord.com/blog/image-thresholding-image-processing/>
- [3] “Connected Component Labeling and Analysis,” PyImageSearch tutorial. [Online]. Available: <https://pyimagesearch.com/2021/02/22/opencv-connected-component-labeling-and-analysis/>
- [4] “Connected Component Analysis in Image Processing,” Scaler tutorial. [Online]. Available: <https://www.scaler.com/topics/connected-component-analysis-in-image-processing/>
- [5] “Introduction to image segmentation,” EPFL Imaging field guide. [Online]. Available: https://imaging.epfl.ch/field-guide/sections/image_segmentation/notebooks/segmentation_intro.html